

ERP 마이크로서비스 기반 결재 시스템 구현 보고서

1. 전체 아키텍처 다이어그램 및 설명

1.1 시스템 개요

본 프로젝트는 다음 구성 요소로 이루어진 마이크로서비스 기반 결재 시스템이다.

- API Gateway
- Employee Service
- Approval Request Service
- Approval Processing Service
- Notification Service
- MySQL
- MongoDB
- RabbitMQ
- Kubernetes

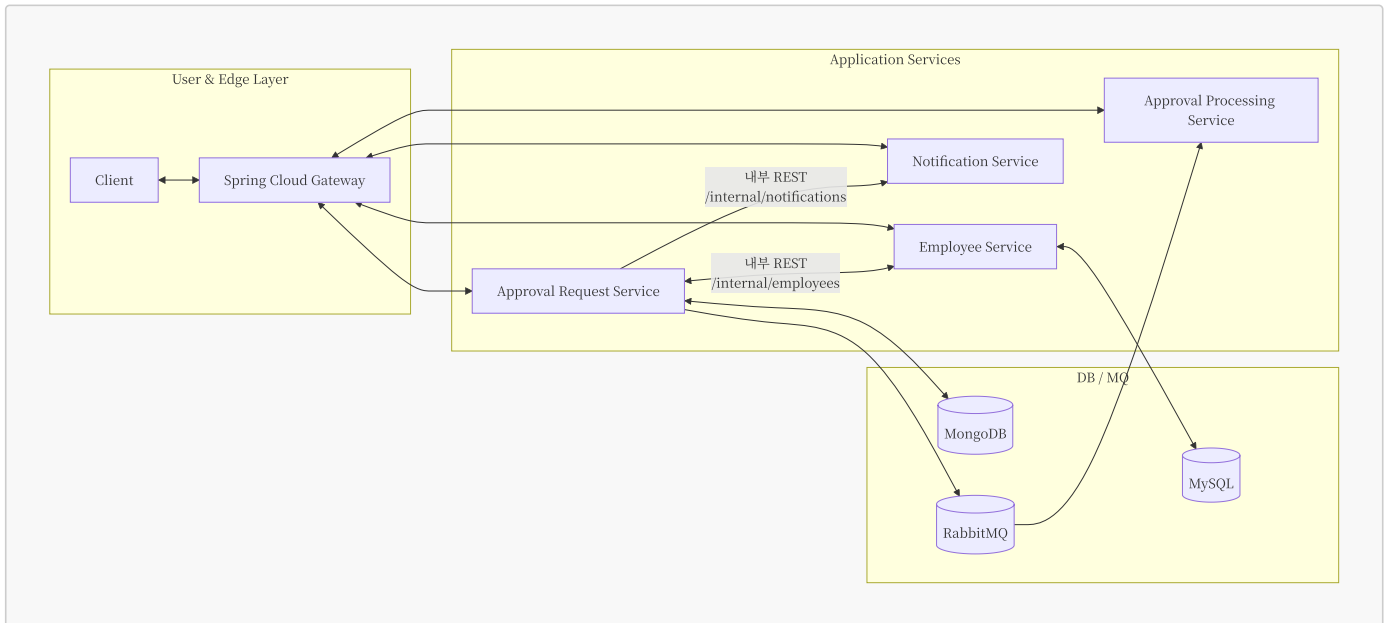
클라이언트는 API Gateway 하나만 바라보고 REST·WebSocket으로 통신하며, Gateway가 JWT를 검증하고 내부 서비스로 요청을 라우팅한다. 결재 처리 파이프라인은 Approval Request Service와 Approval Processing Service 사이의 비동기 메시지(RabbitMQ)로 연결되며, 최종 결과는 Notification Service의 WebSocket을 통해 실시간으로 전달된다.

1.2 서비스 구성 요소 개요

구성 요소	역할	주요 기술
API Gateway	단일 진입점, JWT 검증, 라우팅, 내부 헤더 주입	Spring Cloud Gateway WebFlux
Employee Service	직원 정보 / 권한 관리, 로그인(JWT 발급)	Spring Boot, Spring Data JPA
Approval Request Service	결재 생성·조회, 단계 관리, 결과 반영	Spring Boot, Spring Data MongoDB, Spring AMQP
Approval Processing Service	승인자별 결재 대기열 관리, 승인/반려 처리	Spring Boot, Spring AMQP
Notification Service	WebSocket 세션 관리, 알림 전송	Spring Boot, WebSocket
RabbitMQ	결재 요청/결과 메시지 큐잉 및 라우팅	-
MySQL	직원 마스터 데이터 저장	-
MongoDB	결재 Document 및 단계 정보 저장	-
Kubernetes 클러스터	교내 Solid Cloud 상 Kubernetes 클러스터	-

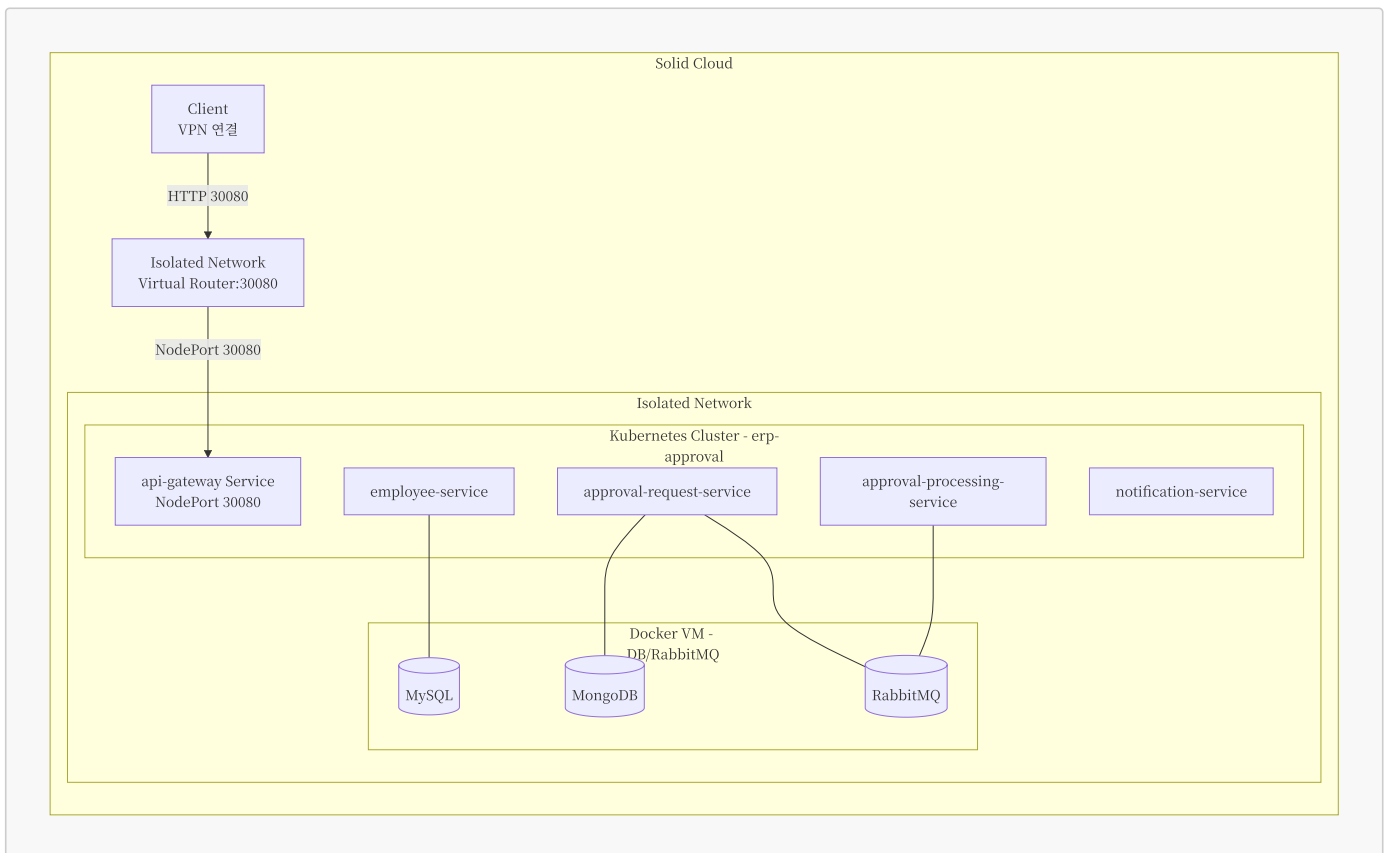
Approval Request Service ↔ Approval Processing Service 사이의 통신은 원래 과제 스펙에서는 gRPC 동기 호출이었으나, **확장 과제 4.2 “비동기 메시지 기반 통신 도입”** 요구에 따라 RabbitMQ 기반 비동기 메시지로 대체하였다. gRPC 계약 자체는 Protobuf(proto 파일)로 유지하면서, 전송 채널만 AMQP로 교체했다.

1.3 전체 아키텍처 다이어그램



1.4 배포 아키텍처 (Kubernetes)

최종 배포는 교내 CloudStack 기반 클라우드인 **Solid Cloud** 상에 구성한 Kubernetes 클러스터를 대상으로 수행했다. 클러스터는 Master Node, Worker Node 두 대의 VM으로 구성되어 있으며, 두 노드 모두 Isolated Network 안에 위치한다.

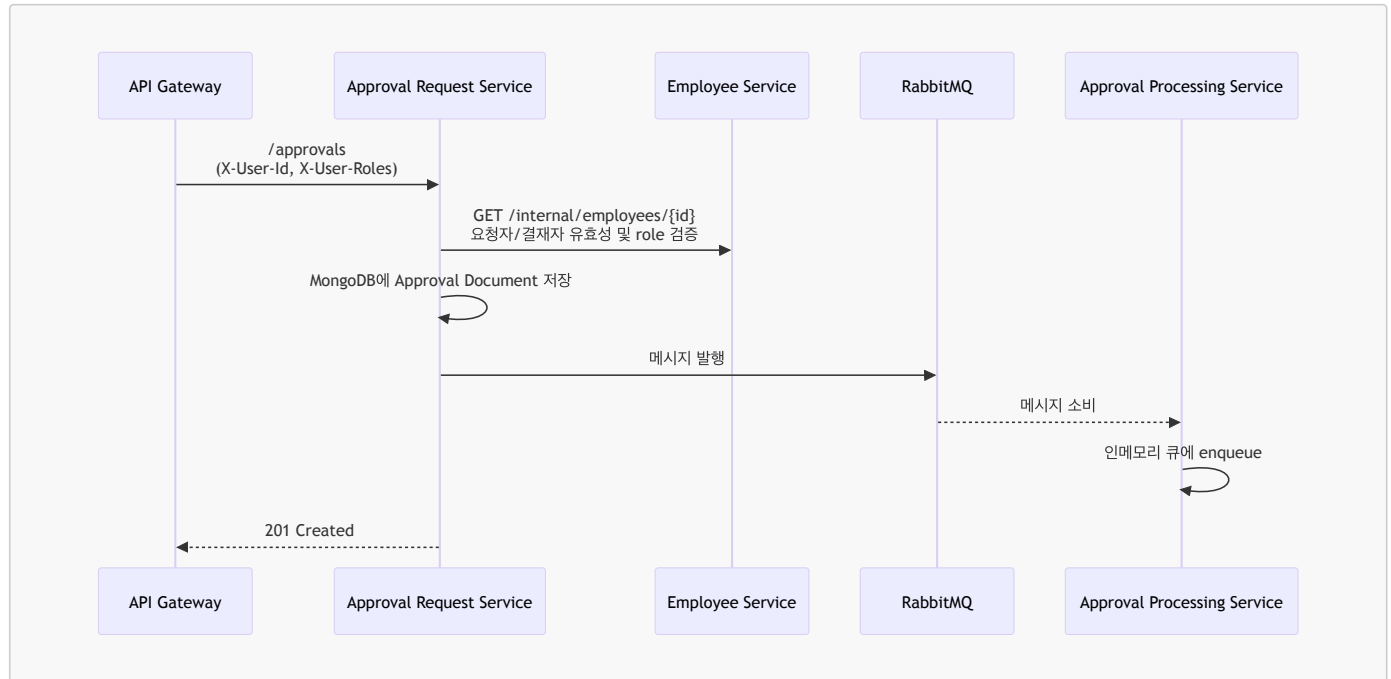


- 네임스페이스: `erp-approval`
- 각 마이크로서비스: `Deployment + Service` 조합으로 배포
 - `api-gateway Service`: `NodePort 30080`
- 공통 설정: `erp-common-config ConfigMap / erp-secret Secret`
- MySQL, MongoDB, RabbitMQ는 Isolated Network 내 별도의 Docker 전용 VM에서 컨테이너로 실행되며, 애플리케이션 Pod는 동일 네트워크 내부에서 Docker VM의 프라이빗 IP와 포트를 통해 접근한다.
- Isolated Network의 가상 라우터에서 `30080 -> 30080` 포트 포워딩을 설정하여, VPN으로 학교 네트워크(Shared Network 대역)에 접속한 후 가상 라우터 IP:30080으로 접근하면 API Gateway로만 진입할 수 있고 나머지 마이크로서비스에는 직접 접근할 수 없다.

2. 서비스 간 호출 흐름도

원래 과제 스펙에서는 Approval Request Service ↔ Approval Processing Service 사이의 통신을 gRPC로 정의하고 있다. 본 구현에서는 **확장 과제 4.2**에 따라 이 구간을 RabbitMQ 기반 비동기 메시지로 대체했으며, 그 외의 부분(REST, WebSocket 구조)은 기본 스펙을 유지했다.

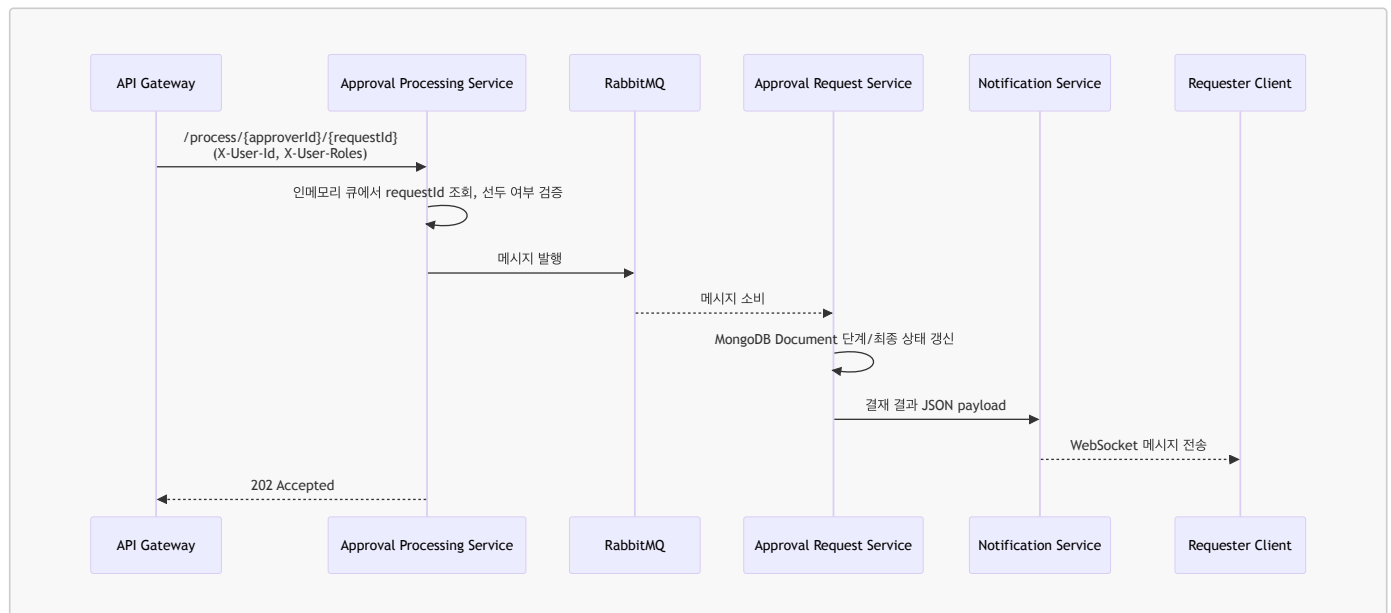
2.1 결재 생성 → 최초 결재 대기열 등록



사용 프로토콜:

- Client(생략됨) ↔ Gateway ↔ ARS/ES: REST(HTTP)
- ARS ↔ APS: 원래 gRPC 동기 호출이었으나, 현재는 Protobuf 메시지를 RabbitMQ(AMQP)로 전달

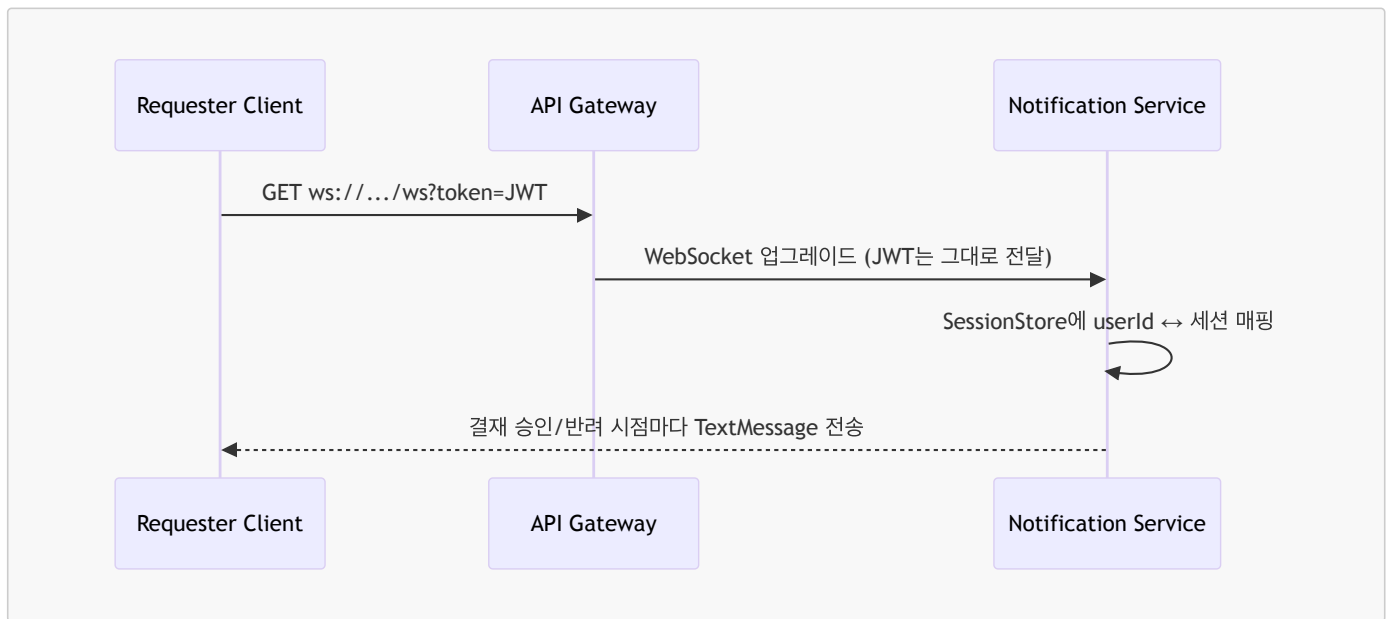
2.2 결재 승인/반려 처리 흐름



사용 프로토콜:

- Client(생략됨) ↔ Gateway ↔ APS: REST(HTTP)
- APS ↔ ARS: Protobuf 메시지를 사용하는 RabbitMQ(AMQP) 기반 비동기 처리
- ARS ↔ NS: REST(HTTP) 내부 호출
- NS ↔ Requester: WebSocket

2.3 WebSocket 알림 흐름



클라이언트는 WebSocket URL의 token 쿼리 파라미터로 JWT를 전달하고, Notification Service는 토큰을 검증한 뒤 직원 ID를 기준으로 세션을 관리한다.

3. REST API 표 정리 (모든 서비스 통합)

아래 표는 API Gateway를 기준으로 외부에 노출되는 REST 엔드포인트를 서비스별로 통합 정리한 것이다. 권한은 JWT의 `roles` 클레임 기준이며, Self-or-Admin 규칙이 있는 경우 별도로 명시했다.

3.1 외부 공개 REST API (Gateway 기준)

서비스	메서드	URI	설명	권한/제약
Employee (Auth)	POST	/auth/login	이메일/비밀번호로 로그인하여 JWT를 발급.	인증 불필요
Employee	POST	/employees	직원 생성. role 및 비밀번호 설정.	ADMIN
Employee	GET	/employees	전체 직원 목록 조회, 부서/직급 필터링.	ADMIN
Employee	GET	/employees/{id}	직원 상세 조회.	ADMIN 또는 본인 (Self-or-Admin)
Employee	PUT	/employees/{id}	직원 정보 수정(department, position, role 등).	ADMIN
Employee	DELETE	/employees/{id}	직원 삭제.	ADMIN
Approval Request	POST	/approvals	결재 요청 생성. JWT sub를 requesterId로 사용.	EMPLOYEE 이상
Approval Request	POST	/approvals/{id}/resend	처리 중단된 결재 요청의 PENDING 단계를 재전송.	요청자·결재자 중 하나 또는 ADMIN
Approval Request	GET	/approvals	(일반) 내가 요청자/결재자인 결재 목록 조회, (ADMIN) 전체 목록 조회.	EMPLOYEE 이상
Approval Request	GET	/approvals/{id}	단일 결재 요청 상세 조회.	요청자·결재자 중 하나 또는 ADMIN
Approval Processing	GET	/process/{approverId}	승인자의 현재 결재 대기열 조회.	APPROVER 이상, Self-or-Admin
Approval Processing	POST	/process/{approverId}/{requestId}	큐 선두 요청에 대해 승인/반려 결과 기록.	APPROVER 이상, Self-or-Admin
Notification	POST	/notifications/{employeeId}	관리자가 특정 직원에게 임의 알림 전송.	ADMIN

3.2 내부 통신용 REST API

내부 서비스끼리만 사용하는 엔드포인트는 `/internal/**`로 분리하였다. API Gateway의 보안 필터와 라우팅에서 `/internal/**` 경로를 차단 하므로, 외부에서는 접근 불가능하고 서비스 간 직접 호출에만 사용된다.

서비스	메서드	URI	사용 주체 / 목적
Employee	GET	/internal/employees/{id}	Approval Request Service가 직원 존재·role 검증용
Notification	POST	/internal/notifications/{employeeId}	Approval Request Service가 결재 결과 알림 전송용

4. gRPC proto 파일 내용

프로젝트는 gRPC 서버/클라이언트 구현 대신 RabbitMQ 기반 비동기 메시지를 사용하지만, 결제 도메인 계약(Contract)은 Protobuf 스키마 (shared-proto/src/main/proto/approval.proto)로 정의하고 있다. Approval Request Service와 Approval Processing Service는 이 Protobuf 정의를 기반으로 동일한 직렬화 포맷을 공유한다.

4.1 Proto 정의

```
syntax = "proto3";

package erp.approval;

option java_package = "erp.shared.proto.approval";
option java_multiple_files = true;

service Approval {
    // Approval Request Service → Approval Processing Service
    rpc RequestApproval (ApprovalRequest) returns (ApprovalResponse);

    // Approval Processing Service → Approval Request Service
    rpc ReturnApprovalResult (ApprovalResultRequest) returns (ApprovalResultResponse);
}

enum StepStatus {
    STEP_STATUS_UNSPECIFIED = 0;
    STEP_STATUS_PENDING = 1;
    STEP_STATUS_APPROVED = 2;
    STEP_STATUS_REJECTED = 3;
}

enum ApprovalResultStatus {
    APPROVAL_RESULT_STATUS_UNSPECIFIED = 0;
    APPROVAL_RESULT_APPROVED = 1;
    APPROVAL_RESULT_REJECTED = 2;
}

message Step {
    int32 step = 1;
    int64 approverId = 2;
    StepStatus status = 3;
}

message ApprovalRequest {
    int64 requestId = 1;
    int64 requesterId = 2;
    string title = 3;
    string content = 4;
    repeated Step steps = 5;
}

message ApprovalResponse {
    string status = 1;
}

message ApprovalResultRequest {
    int64 requestId = 1;
    int32 step = 2;
    int64 approverId = 3;
    ApprovalResultStatus status = 4;
}

message ApprovalResultResponse {
    string status = 1;
}
```

- RequestApproval: Approval Request Service → Approval Processing Service 방향의 결재 요청 전송을 나타내는 RPC 정의였으며, 현재는 동일한 ApprovalRequest 메시지를 RabbitMQ approval.request 라우팅키로 발행하는 데 사용한다.
- ReturnApprovalResult: Approval Processing Service → Approval Request Service 방향의 결과 회신용 RPC 정의였으며, 현재는 ApprovalResultRequest 메시지를 RabbitMQ approval.result 라우팅키로 발행하는 데 재사용한다.

4.2 주요 enum 및 메시지 구조

- StepStatus enum
 - STEP_STATUS_UNSPECIFIED: 초기값(사용 금지 대상).
 - STEP_STATUS_PENDING: 아직 처리되지 않은 단계.
 - STEP_STATUS_APPROVED: 해당 단계 승인 완료.
 - STEP_STATUS_REJECTED: 해당 단계에서 반려된 상태.
- ApprovalResultStatus enum
 - APPROVAL_RESULT_STATUS_UNSPECIFIED: 초기값.
 - APPROVAL_RESULT_APPROVED: 승인.
 - APPROVAL_RESULT_REJECTED: 반려.
- Step 메시지
 - int32 step: 1, 2, 3... 형태의 단계 번호.
 - int64 approverId: 해당 단계를 처리할 승인자 직원 ID.
 - StepStatus status: 단계별 상태(PENDING/APPROVED/REJECTED).
- ApprovalRequest 메시지
 - int64 requestId: Approval Request Service에서 발급한 결재 요청 번호.
 - int64 requesterId: 요청자 직원 ID (employees.id).
 - string title: 결재 제목.
 - string content: 결재 내용.
 - repeated Step steps: 단계별 승인자 목록과 상태.
- ApprovalResponse / ApprovalResultResponse
 - string status: "received" 등 처리 상태를 표현하는 단순 문자열.

실제 구현에서는 이 Protobuf 타입들을 사용해 toByteArray() / parseFrom(byte[]) 기반 직렬화/역직렬화를 수행하며, gRPC 채널을 RabbitMQ 메시지 브로커로 교체했을 뿐 “결재 요청/결과”에 대한 데이터 계약은 그대로 유지하고 있다.

5. MySQL 스키마

Employee Service는 모든 직원 정보와 권한, 로그인 정보를 MySQL `employees` 테이블에 저장한다. 스키마는 `employee-service/src/main/resources/schema.sql`에 다음과 같이 정의되어 있다.

```
CREATE TABLE IF NOT EXISTS employees (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  name VARCHAR(100) NOT NULL,  
  department VARCHAR(50) NOT NULL,  
  position VARCHAR(50) NOT NULL,  
  role VARCHAR(20) NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

5.1 컬럼별 역할

- `id`: 각 직원의 고유 식별자. `AUTO_INCREMENT` PK로 설정되어 Approval Request/Processing/Notification 등 다른 서비스에서 직원 식별에 사용한다.
- `email`: 로그인 ID로 사용하는 이메일 주소. `UNIQUE` 제약을 두어 동일 이메일로 중복 가입을 방지한다.
- `name`: 직원 이름.
- `department`: 부서 정보(예: HR, HQ).
- `position`: 직급 정보(예: Manager, Admin).
- `role`: 권한 레벨. `EMPLOYEE`, `APPROVER`, `ADMIN` 중 하나이며, JWT 생성 시 `roles` 클레임을 구성하는 기준이 된다.
- `password_hash`: BCrypt 등으로 암호화된 비밀번호 해시.
- `created_at`: 행 생성 시각으로, 감사(audit) 용도로 사용한다.

5.2 설계 의도

- Role을 별도 테이블이 아닌 단일 문자열 컬럼으로 관리해 스키마를 단순화하고, JWT 생성 시 Role 계층(`EMPLOYEE < APPROVER < ADMIN`)을 애플리케이션 레벨에서 해석하도록 했다.
- Approval Request/Processing/Notification은 직원 상세 정보를 직접 저장하지 않고 `employees.id`와 REST 호출을 통해 필요한 정보를 가져오도록 하여, 직원 마스터 데이터의 단일 소스를 유지했다.

5.3 권한 계층 구조 (EMPLOYEE / APPROVER / ADMIN)

본 시스템의 권한은 `EMPLOYEE → APPROVER → ADMIN`으로 이어지는 단순 계층 구조이며, 상위 Role은 하위 Role의 권한을 모두 포함한다.



역할별 권한은 다음과 같다.

- `EMPLOYEE`: 결재 요청 생성 및 본인이 요청자/결재자로 포함된 결재 건 조회 가능.
- `APPROVER`: `EMPLOYEE` 권한 + 본인에게 할당된 결재 건에 대해 승인/반려 가능.
- `ADMIN`: `APPROVER` 권한 + 직원 생성/수정/삭제 등 직원 관리 기능까지 모두 수행 가능.

6. MongoDB 문서 구조

결재 요청과 단계 정보는 Approval Request Service의 MongoDB 컬렉션 `approvals`에 Document 형태로 저장된다. 먼저 대표적인 Document 예시를 JSON으로 정리하면 다음과 같다.

```
{
  "requestId": 1,
  "requesterId": 10,
  "title": "지출 결의",
  "content": "법인카드 영수증 첨부",
  "steps": [
    {
      "step": 1,
      "approverId": 20,
      "status": "STEP_STATUS_PENDING",
      "updatedAt": "2025-11-30T12:34:56Z"
    },
    {
      "step": 2,
      "approverId": 30,
      "status": "STEP_STATUS_PENDING",
      "updatedAt": null
    }
  ],
  "createdAt": "2025-11-30T12:30:00Z",
  "updatedAt": "2025-11-30T12:30:00Z",
  "finalStatus": "STEP_STATUS_PENDING",
  "version": 0
}
```

6.1 필드별 설명

- `_id`: string
MongoDB 내부 Object ID. 애플리케이션 레벨에서는 주로 `requestId`로 조회한다.
- `requestId`: long
결재 요청 번호. `@Indexed(unique = true)`로 유니크 인덱스를 두어 중복 생성을 방지한다.
- `requesterId`: long
결재를 요청한 직원 ID (`employees.id`). JWT의 `sub`를 기반으로 설정된다.
- `title`: string
결재 제목.
- `content`: string
결재 내용(예: 지출 내역, 설명).
- `steps`: `StepInfo[]`
결재 단계를 담은 배열. 각 요소는 다음 필드를 가진다.
 - `step`: int: 1부터 시작하는 순서 번호.
 - `approverId`: long: 해당 단계를 처리하는 승인자 ID.
 - `status`: `StepStatus`: `STEP_STATUS_PENDING/APPROVED/REJECTED` 중 하나.
 - `updatedAt`: `Instant`: 해당 단계 상태가 마지막으로 갱신된 시각.
- `createdAt`: `Instant`
결재 요청 생성 시각.
- `updatedAt`: `Instant`
Document 전체가 마지막으로 변경된 시각(단계 상태 변경, 최종 결과 확정 등).
- `finalStatus`: `StepStatus`
전체 결재의 최종 상태를 나타낸다.
초기값은 `STEP_STATUS_PENDING`, 모든 단계 승인 시 `STEP_STATUS_APPROVED`, 어느 한 단계라도 반려되면 `STEP_STATUS_REJECTED`로 설정된다.

- `version: long`
@Version 필드로, Spring Data MongoDB의 낙관적 락(Optimistic Lock)을 위한 버전 값이다. 결과 갱신 시 동시 업데이트 충돌을 감지하는 데 사용된다.

6.2 인덱스 설계

- `requester_idx: { requesterId: 1 }`
특정 요청자가 생성한 결재 목록을 빠르게 조회하기 위한 인덱스이다.
- `approver_idx: { 'steps.approverId': 1 }`
어떤 직원이 결재자로 포함된 모든 요청을 효율적으로 조회하기 위한 인덱스이다. Approval Request Service의 “내가 결재자로 포함된 요청 목록” 기능에서 활용된다.

이 구조를 통해 Approval Request Service는 “요청자 기준 목록 조회”, “승인자 기준 목록 조회”, “단계별 상태 변경 및 최종 상태 판단”이라는 핵심 요구사항을 MongoDB 한 컬렉션으로 해결한다.

7. WebSocket 메시지 구조

Notification Service의 WebSocket 채널은 텍스트 기반 메시지 한 종류만 사용하며, 서버는 단순히 문자열 Payload를 그대로 클라이언트 세션에 전달한다. 알림을 발송하는 REST API와 실제 WebSocket 메시지 구조는 다음과 같다.

7.1 REST NotifyRequest 구조

- 요청 바디
NotifyRequest 레코드: `{"payload": "<문자열>"}` 형식.
Notification Service는 이 `payload`를 그대로 해당 직원의 WebSocket 세션에 `TextMessage`로 전송한다.

7.2 결재 결과 알림 Payload 포맷

Approval Request Service는 결재 최종 상태에 따라 다음 두 가지 형태의 JSON Payload를 생성해 Notification Service에 전달한다.

- 승인 완료 시

```
{ "requestId": 1, "result": "approved", "finalResult": "approved" }
```

- 반려 발생 시

```
{
  "requestId": 1,
  "result": "rejected",
  "rejectedBy": 3,
  "finalResult": "rejected"
}
```

각 필드 의미는 다음과 같다.

- `requestId`: 결재 요청 번호.
- `result`: 이번 이벤트의 결과(승인 "approved", 반려 "rejected").
- `rejectedBy`: 반려한 승인자(직원 ID, 승인 시에는 생략).
- `finalResult`: 전체 결재의 최종 결과.

클라이언트는 WebSocket으로 수신한 문자열을 JSON으로 파싱하여, `result` / `finalResult` / `rejectedBy` 등을 기준으로 알림 문구를 구성하거나 UI 상태를 갱신하면 된다.

8. 실행 방법 (빌드 및 실행 명령어, 환경 설정 포함)

최종 배포 대상은 교내 CloudStack 기반 **Solid Cloud**에 구성된 Kubernetes 클러스터이며, 애플리케이션 컨테이너 이미지는 GitHub Actions 워크플로우가 GHCR(GitHub Container Registry)에 빌드·푸시한 이미지를 사용한다. MySQL, MongoDB, RabbitMQ는 Kubernetes 외부에서 Docker 컨테이너로 실행되고, 애플리케이션 Pod는 환경 변수로 주입된 호스트 IP/포트를 통해 해당 컨테이너에 접근한다.

8.1 Docker 인프라(VM) 기동

- Isolated Network 내 별도의 Docker 전용 VM에서 MySQL, MongoDB, RabbitMQ 컨테이너를 기동한다 (포트: 3306 / 27017 / 5672).
 - Docker VM에 `.env.sample`을 복사하여 `.env` 파일을 생성하고, 환경 변수 값(특히 DB/RabbitMQ 접속 정보)을 채운다.
 - Docker VM에서 다음 명령으로 컨테이너 스택을 기동한다.

```
docker compose up -d
```

```
ubuntu@vm-32212874-advanced-programming-lab-docker: ~/dku-ce-erp-microservices
[+] up 8/8
✓ Network dku-ce-erp-microservices_default          Created          0.1s
✓ Volume dku-ce-erp-microservices_rabbitmq_data     Created          0.0s
✓ Volume dku-ce-erp-microservices_mysql_data        Created          0.0s
✓ Volume dku-ce-erp-microservices_mongo_data        Created          0.0s
✓ Container dku-ce-erp-microservices-mysql-1        Created          0.2s
✓ Container dku-ce-erp-microservices-mongo-1        Healthy         11.7s
✓ Container dku-ce-erp-microservices-rabbitmq-1     Created          0.2s
✓ Container dku-ce-erp-microservices-mongo-init-replica-1 Created          0.1s
ubuntu@vm-32212874-advanced-programming-lab-docker:~/dku-ce-erp-microservices$ docker ps -a --format "table {{.ID}}\t{{.Image}}\t{{.Status}}\t{{.Names}}"
CONTAINER ID   IMAGE                                STATUS          NAMES
ef135382ce54   mongo:8.2                          Exited (0) 29 seconds ago    dku-ce-erp-microservices-mongo-init-replica-1
c99286983977   rabbitmq:4.2.1-management          Up 43 seconds (healthy)     dku-ce-erp-microservices-rabbitmq-1
fe647a63fa48   mongo:8.2                          Up 43 seconds (healthy)     dku-ce-erp-microservices-mongo-1
0bd033be10c9   mysql:8                             Up 43 seconds (healthy)     dku-ce-erp-microservices-mysql-1
ubuntu@vm-32212874-advanced-programming-lab-docker:~/dku-ce-erp-microservices$
```

8.2 애플리케이션 빌드 및 이미지 생성

- GitHub Actions 워크플로우가 빌드 후 GHCR(GitHub Container Registry)에 푸시한다.
- 각 서비스별 이미지는 다음과 같다.
 - `ghcr.io/ilcm96/dku-ce-erp-microservices/api-gateway:latest`
 - `ghcr.io/ilcm96/dku-ce-erp-microservices/employee-service:latest`
 - `ghcr.io/ilcm96/dku-ce-erp-microservices/approval-request-service:latest`
 - `ghcr.io/ilcm96/dku-ce-erp-microservices/approval-processing-service:latest`
 - `ghcr.io/ilcm96/dku-ce-erp-microservices/notification-service:latest`

8.3 Kubernetes 리소스 배포(Master Node)

- Kubernetes 클러스터: Solid Cloud 상 Kubernetes 클러스터(Master/Worker 2노드)가 준비되어 있어야 하며, 8.1의 Docker 인프라(VM) 기동이 완료되어 있어야 한다.
- `k8s/config/secret.sample.yaml`를 `secret.yaml`로 변경하고 내용을 실 환경 값으로 수정한다.

```
mv k8s/config/secret.sample.yaml k8s/config/secret.yaml
```

- `k8s/config/configmap.yaml` 파일에서 MySQL/MongoDB/RabbitMQ 접속을 위한 IP를 Docker VM의 프라이빗 IP로 맞춘다.

- 이후 Master Node에서 다음 명령으로 Kubernetes 리소스를 배포한다.

```
kubectl apply -f k8s/
kubectl apply -f k8s/config/
kubectl apply -f k8s/apps/
```

- 네임스페이스와 ConfigMap/Secret을 먼저 생성한 뒤, 각 서비스 Deployment/Service를 배포한다.
- 배포 후 `kubectl get pods -n erp-approval`로 Pod 상태를 확인한다.

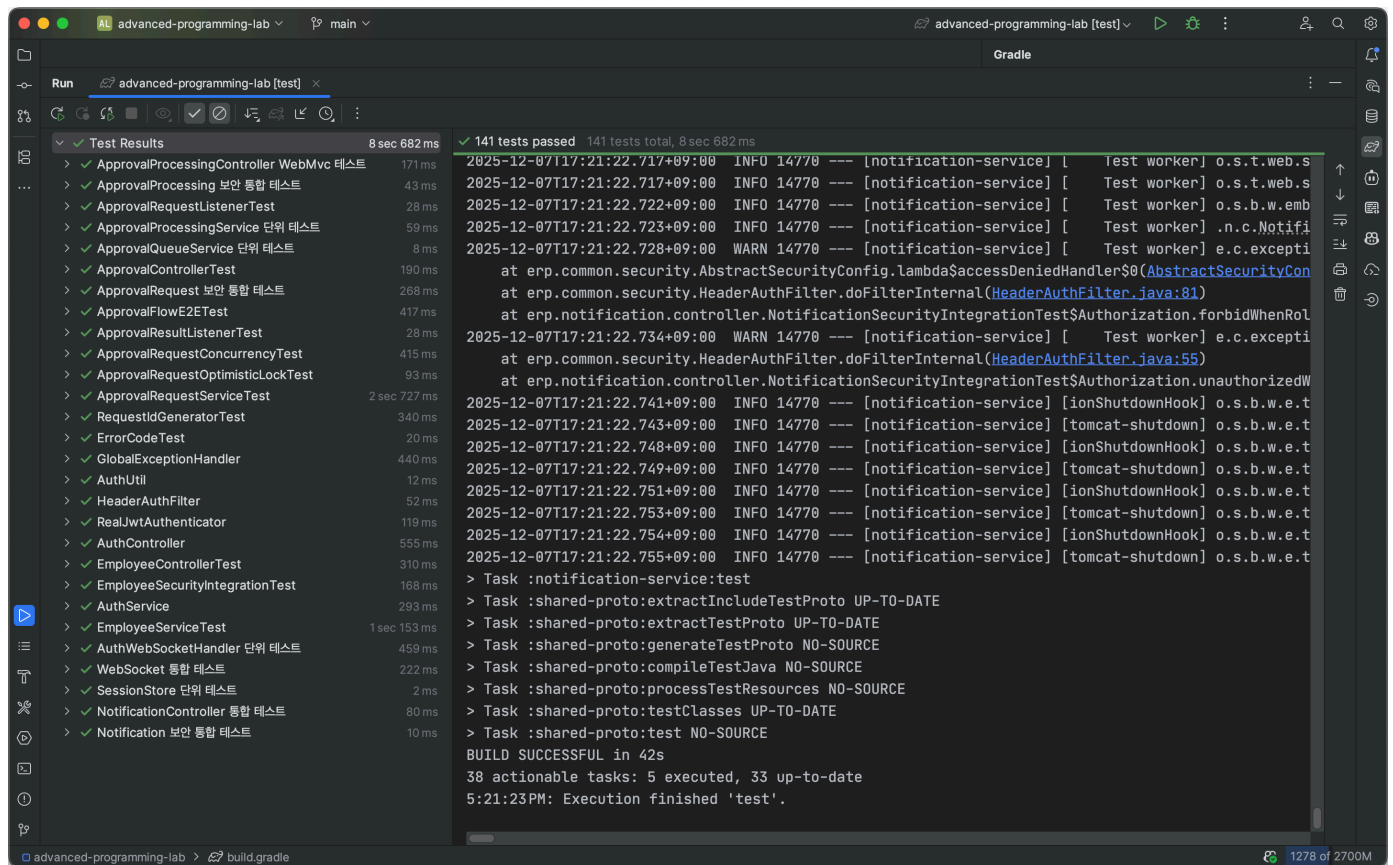
```
ubuntu@vm-32212874-advanced-programming-lab-master:~/dku-ce-erp-microservices$ kubectl apply -f k8s/
namespace/erp-approval created
ubuntu@vm-32212874-advanced-programming-lab-master:~/dku-ce-erp-microservices$ kubectl apply -f k8s/config/
configmap/erp-common-config created
secret/erp-common-secret created
ubuntu@vm-32212874-advanced-programming-lab-master:~/dku-ce-erp-microservices$ kubectl apply -f k8s/apps/
service/api-gateway created
deployment.apps/api-gateway created
service/approval-processing-service created
deployment.apps/approval-processing-service created
service/approval-request-service created
deployment.apps/approval-request-service created
service/employee-service created
deployment.apps/employee-service created
service/notification-service created
deployment.apps/notification-service created
ubuntu@vm-32212874-advanced-programming-lab-master:~/dku-ce-erp-microservices$ kubectl get pod -n erp-approval -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
api-gateway-6d54dfdd4b-f56n4        1/1     Running   0           86s   10.1.200.196    vm-32212874-advanced-programming-lab-worker   <none>           <none>
approval-processing-service-6fb7674b78-27n6s  1/1     Running   0           86s   10.1.200.197    vm-32212874-advanced-programming-lab-worker   <none>           <none>
approval-request-service-b6774678d-8rrn2     1/1     Running   0           86s   10.1.238.197    vm-32212874-advanced-programming-lab-master   <none>           <none>
employee-service-6959759f45-mp5p4           1/1     Running   0           86s   10.1.200.198    vm-32212874-advanced-programming-lab-worker   <none>           <none>
notification-service-7ccffcbdb-sr5kg         1/1     Running   0           86s   10.1.238.198    vm-32212874-advanced-programming-lab-master   <none>           <none>
ubuntu@vm-32212874-advanced-programming-lab-master:~/dku-ce-erp-microservices$ kubectl get svc -n erp-approval -o wide
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP  PORT(S)          AGE   SELECTOR
api-gateway         NodePort      10.152.183.194   <none>        8080:38080/TCP   119s  app=api-gateway
approval-processing-service  ClusterIP    10.152.183.20   <none>        8080/TCP         119s  app=approval-processing-service
approval-request-service  ClusterIP    10.152.183.140  <none>        8080/TCP         118s  app=approval-request-service
employee-service      ClusterIP     10.152.183.33   <none>        8080/TCP         118s  app=employee-service
notification-service    ClusterIP     10.152.183.243  <none>        8080/TCP         118s  app=notification-service
```

8.4 접속 방법

- API Gateway: `http://<ISOLATED_NETWORK_IP>:30080`
 - REST API: `/auth`, `/employees`, `/approvals`, `/process`, `/notifications`
 - WebSocket: `ws://<ISOLATED_NETWORK_IP>:30080/ws?token=<JWT>`

9. 테스트 시나리오 (결재 승인, 반려, 동시성 등)

본 프로젝트의 테스트는 Spring과 JUnit 5를 활용하여 작성하였으며, 공통 보안/에러 처리 → 각 서비스별 단위·통합 테스트 → 결재 승인/반려 전체 플로우 E2E 테스트 → 동시성·락·재시도 시나리오 순으로 구성되어 있으며 총 141개의 테스트 케이스로 어플리케이션의 동작을 검증한다.



아래 표는 스프링 기반 테스트 클래스들을 기준으로 핵심 시나리오를 정리한 것이다.

9.1 서비스별 단위·통합 테스트 개요

구분	주요 테스트 클래스	시나리오 요약	유형
공통 코어	HeaderAuthFilterTest, AuthUtilTest, RealJwtAuthenticatorTest, ErrorCodeTest, GlobalExceptionHandlerTest	헤더 기반 인증 필터, JWT 해석, 에러 코드·예외 매핑이 일관된 형태(HTTP 상태/에러 응답)로 동작하는지 검증	단위 테스트
Employee Service	AuthServiceTest, EmployeeServiceTest, AuthControllerTest, EmployeeControllerTest, EmployeeSecurityIntegrationTest	로그인(JWT 발급) 성공/실패, 직원 CRUD, ADMIN/Self-or-Admin 권한 체크, 401/403 예외 매핑 검증	통합 + WebMvc + 보안
Approval Request	ApprovalRequestServiceTest, ApprovalControllerTest, ApprovalSecurityIntegrationTest, RequestIdGeneratorTest, ApprovalResultListenerTest	결재 생성/조회/재전송, 단계 검증, 요청자·결재자 권한, 고유 requestId 생성, 결과 메시지 수신 처리 검증	통합 + WebMvc + 단위
Approval Processing	ApprovalProcessingServiceTest, ApprovalQueueServiceTest, ApprovalProcessingControllerTest, ApprovalProcessingSecurityIntegrationTest, ApprovalRequestListenerTest	승인자별 결재 대기열 정렬·조회, 큐 선두 요청만 처리, 결과 메시지 발행 및 재시도, 보안·예외 매핑 검증	단위 + WebMvc + 보안
Notification	NotificationControllerTest, NotificationSecurityIntegrationTest, AuthWebSocketHandlerTest, SessionStoreTest, NotificationWebSocketIntegrationTest	알림 REST API 권한 (ADMIN-only), WebSocket 접속/세션 관리, JWT 기반 토큰 인증, 메시지 브로드캐스트 검증	단위 + 통합 + WebSocket

9.2 결재 승인/반려 플로우 시나리오

Approval Request Service ↔ Approval Processing Service ↔ Notification Service까지 이어지는 “결재 생성 → 단계별 승인/반려 → 알림 발송” 플로우는 E2E 테스트와 각 서비스 단위 테스트로 함께 검증한다.

시나리오	관련 테스트	검증 포인트
두 단계 승인 후 최종 승인 알림	ApprovalFlowE2ETest. 결재_두 단계_승인_알림까지_수신한다	요청자가 REST로 결재 생성 → 1단계 승인 시 2단계 승인자의 큐로 이동 → 2단계 승인 시 MongoDB 최종 상태 APPROVED·큐 비움·요청자 WebSocket 알림 확인
단일 단계 반려 후 최종 거부 알림	ApprovalFlowE2ETest. 단일 단계_반려시_최종 거부_알림이_전송된다	단일 승인자가 REJECTED로 처리하면 최종 상태가 REJECTED로 확정되고, 반려자 정보(rejectedBy)를 포함한 WebSocket 알림이 전송되는지 검증
잘못된 단계 번호 (건너뛰기/역순) 요청 시 400 응답	ApprovalFlowE2ETest. 단계 번호가_역순이면 _BAD_REQUEST를_반환한다, ApprovalRequestServiceTest의 단계 검증 케이스들	1→3처럼 단계가 건너뛰거나 역순인 요청을 생성할 경우 APPROVAL_REQUEST_INVALID_STEP 에러 코드와 함께 400 BAD_REQUEST가 반환되는지 검증
다른 승인자가 처리 시 NOT_FOUND 및 큐 유지	ApprovalFlowE2ETest. 다른_승인자가_처리하면 _NOT_FOUND와_대기열_유지가_발생한다, ApprovalProcessingServiceTest	승인 대상 approverId와 다른 사용자가 처리 요청 시 APPROVAL_PROCESS_NOT_FOUND(404) 응답, 원래 승인자의 큐는 그대로 유지되는지 확인
승인자 큐 조회/처리 시 권한 검증	ApprovalProcessingServiceTest.getQueue, ApprovalProcessingControllerTest, ApprovalProcessingSecurityIntegrationTest	ADMIN은 누구나의 큐를 조회 가능, APPROVER는 자신의 큐만 조회·처리 가능, EMPLOYEE/미인증 사용자는 401/403으로 차단되는지 검증

9.3 동시성·락·재시도 시나리오

동시에 여러 승인/반려 요청이 들어오는 상황, MongoDB 낙관적 락 충돌, RabbitMQ 메시지 발행 실패 등에 대비한 방어 로직은 다음 테스트들로 검증한다.

영역	관련 테스트	시나리오 및 검증 포인트
동일 단계 다중 승인 동시 요청	ApprovalRequestConcurrencyTest.concurrentApproveSameStep	동일 결재 요청의 특정 단계에 대해 다수 스레드가 동시에 APPROVED를 전송했을 때, DB에 해당 스텝이 한 번만 APPROVED로 반영되고 다음 스텝은 PENDING 유지, RabbitMQ 메시지는 1회만 발행되는지 검증
동일 단계 승인/반려 Race	ApprovalRequestConcurrencyTest.concurrentApproveAndRejectSameStep	같은 단계에 APPROVED/REJECTED 요청이 섞여 동시에 들어오는 경우, 최종 결과가 APPROVED 또는 REJECTED 중 하나로만 확정되고 Notification은 한 번만 전송되는지 검증
다음 단계가 먼저 처리 요청	ApprovalRequestConcurrencyTest.concurrentNextStepBeforeCurrent	1단계가 처리되지 않은 상태에서 2단계 승인 요청이 동시에 여러 번 들어올 때, 모든 요청이 APPROVAL_PROCESS_INVALID_STATUS로 거절되고 문서 상태·큐·알림·메시지 발행이 전혀 변경되지 않는지 검증
MongoDB 낙관적 락 재시도	ApprovalRequestOptimisticLockTest.낙관적 락_충돌시_재시도 후_성공하면_예외없이_종료한다	OptimisticLockingFailureException이 1회 발생 후 재시도에서 성공하는 케이스에서, 설정된 재시도 횟수 내에서 save 재시도가 수행되고 최종적으로 예외 없이 성공하며 알림이 1회만 전송되는지 검증
낙관적 락 반복 충돌 → CONFLICT	ApprovalRequestOptimisticLockTest.낙관적 락이_계속되면_CONFLICT_예외를_던진다	재시도 횟수를 초과하도록 매번 낙관적 락 충돌이 발생하는 경우 APPROVAL_PROCESS_CONFLICT 에러 코드로 CustomException 이 발생하고 save 호출 횟수가 설정값과 일치하는지 검증

영역	관련 테스트	시나리오 및 검증 포인트
Approval 요청 발행 재시도	ApprovalRequestServiceTest.CallProcessingWithRetry	Approval Request 생성 시 RabbitMQ 발행이 처음에 실패할 때 설정한 횟수만큼 재시도되고, 첫 호출 실패 후 두 번째 호출에서 성공하는 경우와 모든 시도 실패 시 RuntimeException 이 전파되는지 검증
Approval 결과 발행 재시도	ApprovalProcessingServiceTest.retryWhenPublishFails	승인/반려 처리 후 결과 메시지 발행이 실패하는 상황에서 결과 메시지 발행을 최대 횟수만큼 재시도하고, 재시도 후 정상 처리되는지 검증
AMQP Listener 예외 처리	ApprovalRequestListenerTest, ApprovalResultListenerTest	MQ에서 수신한 byte[]가 역직렬화 실패이거나 비즈니스 예외(CustomException)를 던지는 경우, 역직렬화 실패 시 AmqpRejectAndDontRequeueException 으로 DLQ 전송되고 비즈니스 예외는 삼키고 ACK 처리되는지 검증

9.4 인증/인가 및 WebSocket 시나리오

JWT·헤더 기반 인증 및 WebSocket 세션 관리는 다음과 같은 테스트로 검증한다.

시나리오	관련 테스트	검증 포인트
헤더 기반 인증 필터	JwtAuthFilterTest	X-User-Id/X-User-Roles 헤더 누락 시 AUTH_TOKEN_MISSING 발생, 유효 헤더일 때 SecurityContext에 사용자가 저장되는지 확인
REST 엔드포인트 권한 체크	EmployeeSecurityIntegrationTest, ApprovalSecurityIntegrationTest, ApprovalProcessingSecurityIntegrationTest, NotificationSecurityIntegrationTest	토큰 누락 시 401, 권한 부족(EMPLOYEE 등) 시 403, ADMIN/APPROVER에게만 특정 API가 허용되는지 검증
WebSocket 토큰 인증 및 세션 관리	AuthWebSocketHandlerTest, NotificationWebSocketIntegrationTest	WebSocket token 쿼리 파라미터로 JWT를 전달했을 때 세션이 SessionStore에 등록되고, 누락 시 AUTH_TOKEN_MISSING·접속 종료 처리 여부
특정 사용자에게 실시간 알림 전송	NotificationWebSocketIntegrationTest.connectWithJwtToken	Notification Service에서 sessionStore.sendTo(userId, message) 호출 시 해당 사용자 WebSocket 클라이언트가 메시지를 수신하는지 검증

위 시나리오들을 통해 “결제 생성 → 승인/반려 → 최종 상태 확정 → 알림 발송”의 전체 흐름뿐 아니라, 권한·에러 응답·동시성·메시지 재시도·락 충돌까지 실제 운영 환경에서 발생할 수 있는 주요 케이스들을 자동화 테스트로 보장하고 있다.

10. 실행 화면 스크린샷

10.1 공통 실행 환경 및 초기 세팅

아래 스크린샷은 두 가지 결재 시나리오(1차 승인 → 2차 승인, 1차 승인 → 2차 반려)를 실행하기 위한 공통 준비 과정을 보여준다.

- 환경 변수 설정
 - Postman 환경에 API Gateway 기본 URL, 로그인용 계정 정보, WebSocket 접속용 토큰 등을 설정하여 이후 요청이 동일 환경에서 재현 가능하도록 구성했다.

```
advanced-programming-lab main
) export BASE_URL="http://10.0.1.56:30080"
export ADMIN_EMAIL="admin@example.com"
export ADMIN_PASSWORD="password"
export EMP_EMAIL="employee@example.com"
export EMP_PASSWORD="employee123!"
export APPROVER1_EMAIL="approver1@example.com"
export APPROVER1_PASSWORD="approver123!"
export APPROVER2_EMAIL="approver2@example.com"
export APPROVER2_PASSWORD="approver456!"

advanced-programming-lab main
)
```

- 관리자 로그인 및 토큰 발급
 - POST /auth/login 엔드포인트를 통해 ADMIN 계정으로 로그인하고, 이후 직원/승인자 생성에 사용할 관리자용 JWT를 발급받는다.

```
advanced-programming-lab main
) ADMIN_TOKEN=$(curl -s -X POST "${BASE_URL}/auth/login" \
  -H "Content-Type: application/json" \
  -d '{"email":"'${ADMIN_EMAIL}'","password":"'${ADMIN_PASSWORD}'"}' \
  | jq -r '.token')

advanced-programming-lab main
) echo "ADMIN_TOKEN=${ADMIN_TOKEN}"
ADMIN_TOKEN=eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxIiwicm9sZXMiOiJRU1QTE9ZRUUUlCJBUBFBST1ZFUiIsIkFETU0lOIsImIhdCI6MTc2NTA5Njc3NCwiZmxhZCI6IjoxNzY1MTAwMzc0fQ.zbGMNUV4xN270K05fZYMTSTRmKfZyGRljtnPfIB4EMw

advanced-programming-lab main
)
```


- 직원/승인자 계정 생성

- Employee Service API를 통해 일반 직원(Employee) 1명과 Approver 역할을 가진 승인자 2명을 생성한다.

```
advanced-programming-lab main
) EMPLOYEE_ID=$(curl -s -X POST "${BASE_URL}/employees" \
-H "Authorization: Bearer ${ADMIN_TOKEN}" \
-H "Content-Type: application/json" \
-d "{
  \"email\": \"${EMP_EMAIL}\",
  \"name\": \"Employee User\",
  \"department\": \"Sales\",
  \"position\": \"Staff\",
  \"role\": \"EMPLOYEE\",
  \"password\": \"${EMP_PASSWORD}\"
}" | jq -r '.id')
```

```
advanced-programming-lab main
) echo "EMPLOYEE_ID=${EMPLOYEE_ID}"
EMPLOYEE_ID=2
```

```
advanced-programming-lab main
)
```

```
advanced-programming-lab main
) APPROVER1_ID=$(curl -s -X POST "${BASE_URL}/employees" \
-H "Authorization: Bearer ${ADMIN_TOKEN}" \
-H "Content-Type: application/json" \
-d "{
  \"email\": \"${APPROVER1_EMAIL}\",
  \"name\": \"Approver One\",
  \"department\": \"HQ\",
  \"position\": \"Manager\",
  \"role\": \"APPROVER\",
  \"password\": \"${APPROVER1_PASSWORD}\"
}" | jq -r '.id')
```

```
advanced-programming-lab main
) APPROVER2_ID=$(curl -s -X POST "${BASE_URL}/employees" \
-H "Authorization: Bearer ${ADMIN_TOKEN}" \
-H "Content-Type: application/json" \
-d "{
  \"email\": \"${APPROVER2_EMAIL}\",
  \"name\": \"Approver Two\",
  \"department\": \"HQ\",
  \"position\": \"Director\",
  \"role\": \"APPROVER\",
  \"password\": \"${APPROVER2_PASSWORD}\"
}" | jq -r '.id')
```

```
advanced-programming-lab main
) echo "APPROVER1_ID=${APPROVER1_ID}"
APPROVER1_ID=3
```

```
) echo "APPROVER2_ID=${APPROVER2_ID}"
APPROVER2_ID=4
```

```
)
```

- 로그인 검증 및 기본 세션 준비

- 생성된 Employee Approver 계정 각각에 대해 로그인 요청을 전송하여 JWT가 정상 발급되는지 확인하고, 이후 결재 요청/승인에 사용할 토큰을 준비한다.

```
YSM-Macbook-Air

advanced-programming-lab main
) EMP_TOKEN=$(curl -s -X POST "${BASE_URL}/auth/login" \
-H "Content-Type: application/json" \
-d "{\"email\":\"${EMP_EMAIL}\",\"password\":\"${EMP_PASSWORD}\"}" \
| jq -r '.token')

advanced-programming-lab main
) APPROVER1_TOKEN=$(curl -s -X POST "${BASE_URL}/auth/login" \
-H "Content-Type: application/json" \
-d "{\"email\":\"${APPROVER1_EMAIL}\",\"password\":\"${APPROVER1_PASSWORD}\"}" \
| jq -r '.token')

advanced-programming-lab main
) APPROVER2_TOKEN=$(curl -s -X POST "${BASE_URL}/auth/login" \
-H "Content-Type: application/json" \
-d "{\"email\":\"${APPROVER2_EMAIL}\",\"password\":\"${APPROVER2_PASSWORD}\"}" \
| jq -r '.token')

advanced-programming-lab main
) echo "EMP_TOKEN=${EMP_TOKEN}"
EMP_TOKEN=eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIyIiwicm9sZXMiOiIsiRU1QTE9ZRUUuIiwiaWF0IjoxNzY1MDk2ODQ3LCJleHAiOiJlE3NjUwMDA0NDd9.76ebvs58Ihw8YemASKSkAz4smaT_qf2ekUq9WJZfPY4

advanced-programming-lab main
) echo "APPROVER1_TOKEN=${APPROVER1_TOKEN}"
APPROVER1_TOKEN=eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIzIiwicm9sZXMiOiIsiRU1QTE9ZRUUuIiwiaWF0IjoxNzY1MDk2ODQ3LCJleHAiOiJlE3NjUwMDA0NDd9.76ebvs58Ihw8YemASKSkAz4smaT_qf2ekUq9WJZfPY4

advanced-programming-lab main
) echo "APPROVER2_TOKEN=${APPROVER2_TOKEN}"
APPROVER2_TOKEN=eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0Iiwicm9sZXMiOiIsiRU1QTE9ZRUUuIiwiaWF0IjoxNzY1MDk2ODQ3LCJleHAiOiJlE3NjUwMDA0NDd9.76ebvs58Ihw8YemASKSkAz4smaT_qf2ekUq9WJZfPY4

advanced-programming-lab main
)

```

- 결재 요청 생성

- 일반 직원 계정으로 POST /approvals를 호출하여 1차/2차 승인자를 모두 포함하는 다단계 결재 요청을 생성한다. 이 요청은 두 시나리오에서 공통으로 사용되며, 최초 상태는 모든 단계가 PENDING이다.

```
YSM-Macbook-Air

advanced-programming-lab main
) REQUEST_ID=$(curl -s -X POST "${BASE_URL}/approvals" \
-H "Authorization: Bearer ${EMP_TOKEN}" \
-H "Content-Type: application/json" \
-d "{\"title\": \"노트북 구매 품의\", \"content\": \"개발용 노트북 구매 요청\", \"steps\": [ { \"step\": 1, \"approverId\": ${APPROVER1_ID} }, { \"step\": 2, \"approverId\": ${APPROVER2_ID} } ] }" \
| jq -r '.requestId')

advanced-programming-lab main
) echo "REQUEST_ID=${REQUEST_ID}"
REQUEST_ID=1

advanced-programming-lab main
)

```

10.2 시나리오 1: 1차 승인 → 2차 승인 (최종 승인)

이 시나리오에는 두 명의 승인자가 순서대로 모두 승인하여 결재 요청이 최종적으로 APPROVED 상태가 되는 흐름을 검증한다.

- 결재 요청 생성 및 초기 상태 확인
 - 생성된 결재 요청을 조회했을 때, 단계별 승인자 정보와 함께 `steps.status = PENDING`, `finalStatus = PENDING` 인 초기 상태를 확인한다.

```
YSM-Macbook-Air

advanced-programming-lab main
) curl -s -X GET "${BASE_URL}/approvals" \
  -H "Authorization: Bearer ${EMP_TOKEN}" | jq
[
  {
    "requestId": 1,
    "id": "69353db4217b653db62a5a8f",
    "requesterId": 2,
    "title": "노트북 구매 품의",
    "content": "개발용 노트북 구매 요청",
    "steps": [
      {
        "step": 1,
        "approverId": 3,
        "status": "STEP_STATUS_PENDING"
      },
      {
        "step": 2,
        "approverId": 4,
        "status": "STEP_STATUS_PENDING"
      }
    ],
    "finalStatus": "STEP_STATUS_PENDING",
    "createdAt": "2025-12-07T08:41:24.822Z",
    "updatedAt": "2025-12-07T08:41:24.822Z"
  }
]

advanced-programming-lab main
)
```

- 1차 승인자의 대기열 확인
 - GET `/process/{approverId}`로 1차 승인자와 2차 승인자의 대기열을 조회했을 때, 최초에는 1차 승인자의 큐에만 결재 요청이 존재하고 2차 승인자의 큐는 비어 있는 것을 확인한다.

```
YSM-Macbook-Air

advanced-programming-lab main
) curl -s -X GET "${BASE_URL}/process/${APPROVER1_ID}" \
  -H "Authorization: Bearer ${APPROVER1_TOKEN}" | jq
[
  {
    "requestId": 1,
    "requesterId": 2,
    "title": "노트북 구매 품의",
    "content": "개발용 노트북 구매 요청",
    "steps": [
      {
        "step": 1,
        "approverId": 3,
        "status": "STEP_STATUS_PENDING"
      },
      {
        "step": 2,
        "approverId": 4,
        "status": "STEP_STATUS_PENDING"
      }
    ]
  }
]

advanced-programming-lab main
) curl -s -X GET "${BASE_URL}/process/${APPROVER2_ID}" \
  -H "Authorization: Bearer ${APPROVER2_TOKEN}" | jq
[]

advanced-programming-lab main
)
```

- 1차 승인 승인 처리

- 1차 승인자가 POST /process/{approverId}/{requestId}를 status=APPROVED로 호출하면, 메시지가 RabbitMQ를 통해 Approval Request Service로 전달되고 1차 단계의 status가 APPROVED로 변경된다.
- 동시에 1차 승인자의 큐에서는 해당 요청이 사라지고, 2차 승인자의 큐에 새로운 대기 전으로 등장하는 것을 확인한다.

```
advanced-programming-lab main
) curl -s -X POST "${BASE_URL}/process/${APPROVER1_ID}/${REQUEST_ID}" \
-H "Authorization: Bearer ${APPROVER1_TOKEN}" \
-H "Content-Type: application/json" \
-d '{ "status": "APPROVAL_RESULT_APPROVED" }' | jq

advanced-programming-lab main
) curl -s -X GET "${BASE_URL}/process/${APPROVER1_ID}" \
-H "Authorization: Bearer ${APPROVER1_TOKEN}" | jq
[]

advanced-programming-lab main
) curl -s -X GET "${BASE_URL}/process/${APPROVER2_ID}" \
-H "Authorization: Bearer ${APPROVER2_TOKEN}" | jq
[
  {
    "requestId": 1,
    "requesterId": 2,
    "title": "노트북 구매 품의",
    "content": "개발용 노트북 구매 요청",
    "steps": [
      {
        "step": 1,
        "approverId": 3,
        "status": "STEP_STATUS_APPROVED"
      },
      {
        "step": 2,
        "approverId": 4,
        "status": "STEP_STATUS_PENDING"
      }
    ]
  }
]
```

- 2차 승인 승인 처리 및 최종 상태 확인

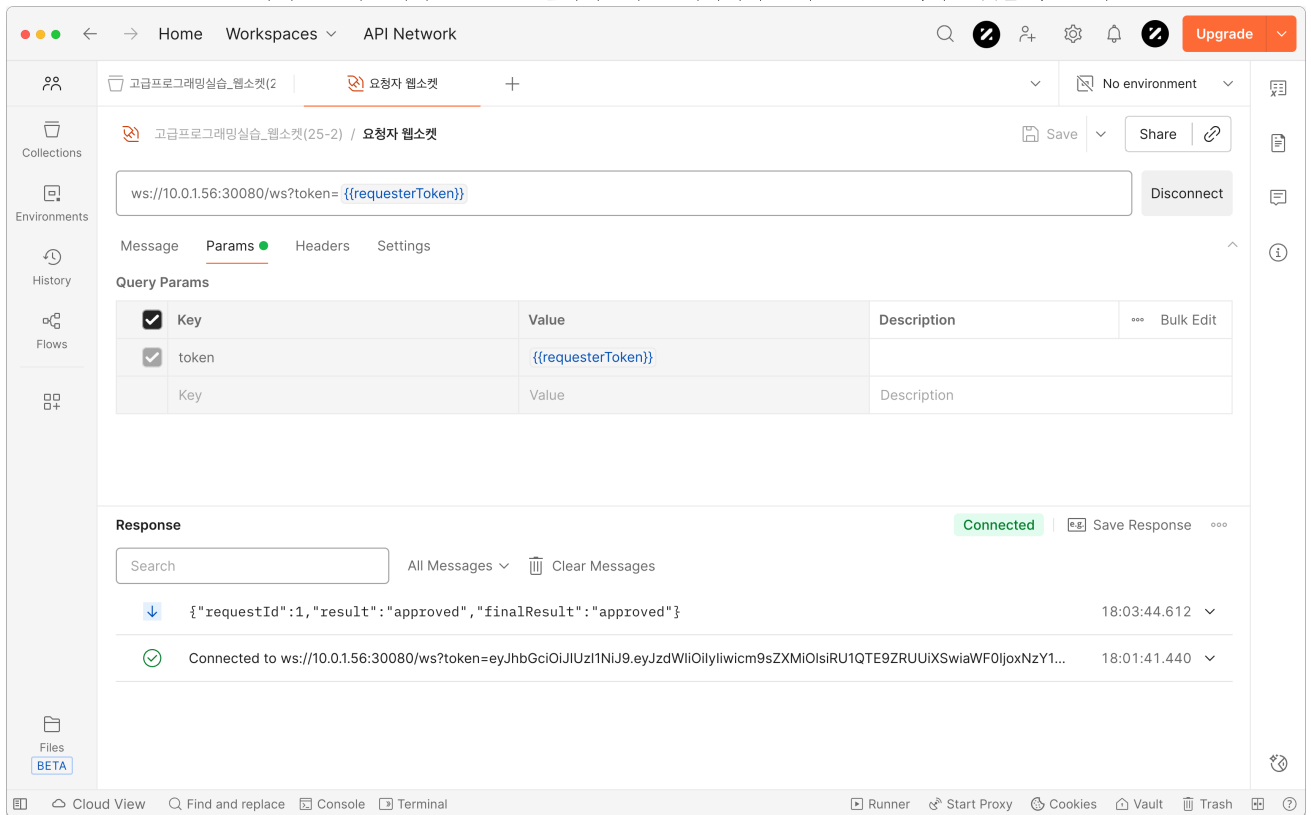
- 2차 승인자도 동일한 API를 status=APPROVED로 호출하면, 두 번째 단계까지 모두 APPROVED가 되어 Approval Request Document의 finalStatus가 APPROVED로 확정된다.
- 이후 요청 상태를 다시 조회했을 때 모든 단계가 APPROVED이며, 더 이상 어떤 승인자의 큐에도 남아 있지 않은 것을 확인한다.

```
advanced-programming-lab main
) curl -s -X POST "${BASE_URL}/process/${APPROVER2_ID}/${REQUEST_ID}" \
-H "Authorization: Bearer ${APPROVER2_TOKEN}" \
-H "Content-Type: application/json" \
-d '{ "status": "APPROVAL_RESULT_APPROVED" }' | jq

advanced-programming-lab main
) curl -s -X GET "${BASE_URL}/process/${APPROVER2_ID}" \
-H "Authorization: Bearer ${APPROVER2_TOKEN}" | jq
[]

advanced-programming-lab main
) curl -s -X GET "${BASE_URL}/approvals/${REQUEST_ID}" \
-H "Authorization: Bearer ${EMP_TOKEN}" | jq
{
  "requestId": 1,
  "id": "693542977989870302b4c714",
  "requesterId": 2,
  "title": "노트북 구매 품의",
  "content": "개발용 노트북 구매 요청",
  "steps": [
    {
      "step": 1,
      "approverId": 3,
      "status": "STEP_STATUS_APPROVED"
    },
    {
      "step": 2,
      "approverId": 4,
      "status": "STEP_STATUS_APPROVED"
    }
  ],
  "finalStatus": "STEP_STATUS_APPROVED",
  "createdAt": "2025-12-07T09:02:15.127Z",
  "updatedAt": "2025-12-07T09:03:44.513Z"
}
```

- WebSocket을 통한 승인 알림 수신
 - 요청자 클라이언트는 Notification Service의 WebSocket에 미리 접속해 있으며, 최종 승인 시점에 해당 직원 ID를 대상으로 승인 알림 메시지(JSON)가 전송된다.
 - Postman WebSocket 클라이언트 화면에서 **APPROVED** 결과가 포함된 메시지가 실시간으로 도착하는 것을 확인한다.



10.3 시나리오 2: 1차 승인 → 2차 반려 (최종 반려)

두 번째 시나리오오는 동일한 구조의 결재 요청에 대해 1차 승인자는 승인하지만 2차 승인자가 반려하여 최종적으로 REJECTED 상태가 되는 흐름을 검증한다.

- 시나리오 2용 결재 요청 생성
 - 앞선 시나리오와 구분하기 위해 새로운 결재 요청을 생성하고, 동일하게 1차·2차 승인자를 모두 설정한다.

```
advanced-programming-lab main
) REQUEST_ID_REJECT=$(curl -s -X POST "${BASE_URL}/approvals" \
-H "Authorization: Bearer ${EMP_TOKEN}" \
-H "Content-Type: application/json" \
-d "{
  \"title\": \"출장비 정산 품의\",
  \"content\": \"출장비 정산 요청\",
  \"steps\": [
    { \"step\": 1, \"approverId\": ${APPROVER1_ID} },
    { \"step\": 2, \"approverId\": ${APPROVER2_ID} }
  ]
}" | jq -r '.requestId')

advanced-programming-lab main
) echo "REQUEST_ID_REJECT=${REQUEST_ID_REJECT}"
REQUEST_ID_REJECT=2

advanced-programming-lab main
)
```

- 1차 승인자의 승인 처리
 - 시나리오 1과 동일하게 1차 승인자가 `status=APPROVED`로 처리하여 1단계가 승인되고, 결재 요청이 2차 승인자의 큐로 이동한다. (화면 구조는 10.2와 동일하므로 별도 캡처는 생략한다.)
- 2차 승인자의 반려 처리 및 최종 상태 확인
 - 2차 승인자가 `POST /process/{approverId}/{requestId}`를 `status=REJECTED`로 호출하면, 해당 단계의 상태가 REJECTED로 변경되고 Approval Request Document의 `finalStatus`가 REJECTED로 확정된다.

- 요청 상태를 조회했을 때 1단계는 APPROVED, 2단계는 REJECTED, 전체 finalStatus는 REJECTED로 표시되는 것을 확인한다.

```

) curl -s -X POST "${BASE_URL}/process/${APPROVER1_ID}/${REQUEST_ID_REJECT}" \
-H "Authorization: Bearer ${APPROVER1_TOKEN}" \
-H "Content-Type: application/json" \
-d '{"status": "APPROVAL_RESULT_APPROVED"}' | jq

advanced-programming-lab main
) curl -s -X POST "${BASE_URL}/process/${APPROVER2_ID}/${REQUEST_ID_REJECT}" \
-H "Authorization: Bearer ${APPROVER2_TOKEN}" \
-H "Content-Type: application/json" \
-d '{"status": "APPROVAL_RESULT_REJECTED"}' | jq

advanced-programming-lab main
) curl -s -X GET "${BASE_URL}/approvals/${REQUEST_ID_REJECT}" \
-H "Authorization: Bearer ${EMP_TOKEN}" | jq
{
  "requestId": 2,
  "id": "693543397989870302b4c715",
  "requesterId": 2,
  "title": "출장비 정산 품의",
  "content": "출장비 정산 요청",
  "steps": [
    {
      "step": 1,
      "approverId": 3,
      "status": "STEP_STATUS_APPROVED"
    },
    {
      "step": 2,
      "approverId": 4,
      "status": "STEP_STATUS_REJECTED"
    }
  ],
  "finalStatus": "STEP_STATUS_REJECTED",
  "createdAt": "2025-12-07T09:04:57.759Z",
  "updatedAt": "2025-12-07T09:05:50.126Z"
}

```

- WebSocket을 통한 반려 알림 수신

- 요청자 클라이언트는 동일하게 WebSocket에 연결된 상태에서, 최종 반려 시점에 REJECTED 결과가 담긴 알림 메시지를 수신한다.
- Postman WebSocket 클라이언트 화면에서 반려 관련 메시지가 실시간으로 도착하는 것을 확인하여 Notification Service와 RabbitMQ·MongoDB 연동이 정상 동작함을 검증한다.

The screenshot shows the Postman WebSocket client interface. The URL bar displays the connection URL: `ws://10.0.1.56:30080/ws?token={{requesterToken}}`. The interface includes tabs for Message, Params, Headers, and Settings. The Params tab is active, showing a query parameter table:

Key	Value	Description
token	{{requesterToken}}	

Below the Params tab, the Response section shows a list of messages received. The first message is a 'rejected' status for requestId 2, received at 18:05:50.066. The second message is an 'approved' status for requestId 1, received at 18:03:44.612. The interface also shows a 'Connected' status at the bottom.

11. 창의 영역 및 확장 구현 (JWT, 비동기 통신, Kubernetes)

11.1 기본 과제 스펙 대비 확장 포인트

과제의 기본 스펙은 “단일 Docker Compose + gRPC 동기 통신 + 간단한 인증 방식”을 가정하고 있다. 본 프로젝트는 이를 다음 세 축으로 확장하였다.

- 인증/인가: Request Body/URL에 requesterId, approverId를 직접 넘기는 방식이 아닌, **JWT 기반 토큰 인증**으로 전환하여 클라이언트가 “누구인지”를 증명하게 했다.
- 서비스 간 통신: Approval Request Service ↔ Approval Processing Service 간 동기 gRPC 대신, **RabbitMQ 기반 비동기 메시지**를 도입하여 큐잉·재시도·내결함성을 확보했다.
- 배포/운영: docker-compose에 머무르지 않고 교내 Solid Cloud 상 **Kubernetes 클러스터**에 실제로 배포·운영할 수 있는 형태로 확장했다.

이 세 요소는 서로 분리된 선택이 아니라, 아래와 같은 방향성에서 하나의 스토리를 형성한다.

- JWT: “요청하는 사용자가 누구인지, 어떤 권한을 가졌는지”를 토큰 한 장으로 표현·검증.
- RabbitMQ: “요청에 대한 처리는 지금 바로 끝나지 않아도 되며, 메시지 단위로 안전하게 위임할 수 있다”는 비동기 파이프라인.
- Kubernetes: “서비스 인스턴스 수나 배포 환경이 바뀌어도 전체 시스템이 안정적으로 동작할 수 있어야 한다”는 운영 관점 확장.

11.2 JWT 기반 인증/인가 설계 심화

11.2.1 토큰 구조 및 클레임 설계

- 토큰의 sub 클레임에는 직원 ID(employees.id)를 그대로 사용하여, 모든 서비스에서 Long 타입의 사용자 식별자를 일관되게 활용할 수 있도록 했다.
- roles 클레임에는 EMPLOYEE, APPROVER, ADMIN 값을 배열 형태로 담아, 하나의 계정이 여러 역할을 동시에 가질 수 있도록 했다.
- JWT 파싱은 RealJwtAuthenticator가 담당하며, 유효하지 않은 토큰/만료 토큰/서명 오류 등을 모두 CustomException + ErrorCode로 변환하여 공통 에러 응답 포맷을 유지한다.

이렇게 하면 각 서비스는 JWT 라이브러리 세부 사용법을 몰라도, AuthUtil을 통해 “현재 사용자 ID”와 “역할 목록”만 가져와 비즈니스 로직을 작성하면 된다.

11.2.2 Gateway 중심의 인증 위임

- 기존 단순 구현에서는 각 서비스가 직접 토큰을 검사하거나, 아예 인증을 고려하지 않고 URL/Body에 userId 값을 넣는 방식에 의존하기 쉽다.
- 본 프로젝트에서는 API Gateway에 GatewayAuthenticationFilter를 두어, **모든 외부 요청이 Gateway를 통과해야만 하는 구조**로 만들었다.
 - 유효한 JWT인 경우에만 내부 서비스로 라우팅하고, X-User-Id, X-User-Roles 헤더를 추가해 전달한다.
 - JWT가 유효하지 않으면 Gateway 단계에서 바로 401/403 응답을 반환하여, 내부 서비스는 “신뢰할 수 있는 헤더만 온다”고 가정할 수 있게 했다.
- 내부 서비스에서는 다시 Spring Security 필터(HeaderAuthFilter)를 통해 Header를 AuthenticatedUser 객체로 변환하고, SecurityContext에 주입한다.

이 구조는 “외부에 노출되는 토큰 검증 로직은 Gateway에 집중시키고, 도메인 서비스는 도메인 로직에만 집중한다”는 마이크로서비스 보안 설계 원칙을 따르도록 한다.

11.2.3 WebSocket 및 내부 API와의 연계

- Notification Service의 WebSocket은 ?token=JWT 쿼리 파라미터로 토큰을 전달받으며, 서버 측에서 토큰을 검증해 userId 기준으로 세션을 관리한다.
 - 이를 통해 “직원 ID를 URL로 넘기는 WebSocket 경로”를 제거하고, WebSocket에서도 REST와 동일한 인증 모델을 적용했다.
- /internal/** 경로는 API Gateway에서 외부 라우팅을 막고, 서비스 간 내부 호출 전용으로만 사용한다.
 - 예: Approval Request Service → Employee Service의 /internal/employees/{id} 호출.
 - 내부 경로는 Gateway를 우회하는 직접 호출이므로, 네트워크 격리와 서비스 간 신뢰를 전제로 동작한다.

11.3 RabbitMQ 비동기 통신 설계 상세

11.3.1 토폴로지 및 메시지 포맷

RabbitMQ 구성은 `ApprovalMessagingConstants`에 상수로 정리하여, 두 서비스가 동일한 토폴로지를 공유한다.

- Exchange: `approval.exchange` (topic 또는 direct)
- Queue: `approval.request.queue`, `approval.result.queue`
- RoutingKey: `approval.request`, `approval.result`
- Payload: gRPC에서 사용하던 Protobuf 메시지(`ApprovalRequest`, `ApprovalResultRequest`)를 그대로 직렬화한 `byte[]`

11.3.2 프로듀서/컨슈머 및 재시도 전략

- Approval Request Service
 - 결재 생성 시 MongoDB에 Document를 저장한 뒤, `ApprovalRequest` 메시지를 생성해 `approval.request` 라우팅키로 발행한다.
 - `callProcessingWithRetry` 메서드가 `approval-request.retry.max-attempts`와 `approval-request.retry.backoff-millis` 설정값을 기반으로 재시도 로직을 수행한다.
- Approval Processing Service
 - `ApprovalRequestListener`가 `approval.request.queue`를 구독하여, 수신된 결재 요청을 승인자별 인메모리 큐에 enqueue 한다.
 - 승인/반려 처리 시 `ApprovalResultRequest`를 만들어 `approval.result` 라우팅키로 전송하며, 이 과정에서도 동일한 재시도 전략을 사용한다.
- Approval Request Service(결과 수신)
 - `ApprovalResultListener`가 `approval.result.queue`를 구독하여, MongoDB Document의 단계 상태 및 `finalStatus`를 갱신하고 Notification Service로 알람을 발송한다.

재시도 로직은 네트워크 순간 장애나 RabbitMQ 일시 다운 같은 상황을 고려한 것이며, *“최소 한 번(at-least-once) 전달”*을 지향한다.

11.3.3 동시성·락 및 멍등성 고려

- Approval Request Document에는 `@Version` 필드를 두어 낙관적 락을 적용하고, 메시지 처리 중 동시 업데이트 충돌이 발생하면 예외를 통해 감지한다.
- 결과 메시지를 처리하는 쪽에서는 “이미 처리된 상태인지”를 검증하여, 컨슈머 중복 실행이나 재전송이 있더라도 **최종 상태가 두 번 반영되지 않도록** 방어한다.
- 승인자별 인메모리 큐는 단일 스레드 기반으로 동작하며, 각 승인자에 대해 “선입선출(FIFO) + 큐 선두 요청만 처리 가능” 규칙을 유지한다.

이러한 설계는 잠금을 DB 테이블 레벨이 아니라 “문서 버전 + 큐 순서” 조합으로 처리하도록 구성했다.

11.4 Kubernetes 기반 운영 시나리오 및 확장 전략

11.4.1 설정/이미지/리소스 분리

- 설정 분리
 - JWT 시크릿, DB/RabbitMQ 접속 정보, 내부 서비스 URL 등은 `erp-common-config` ConfigMap과 `erp-secret` Secret으로 분리했다.
 - 코드에는 “환경 변수 이름”만 남기고, 실제 값은 배포 환경에서 주입함으로써 **환경에 따라 설정만 바꾸어 재배포**할 수 있도록 했다.
- 이미지 관리
 - 모든 서비스 이미지는 GHCR에 `ghcr.io/ilcm96/dku-ce-erp-microservices/<service>:latest` 형태로 관리한다.
 - GitHub Actions 워크플로우가 빌드 → 푸시까지 자동화하도록 구성하여, 로컬 개발 환경과 배포 환경을 분리했다.
- 리소스 및 배포 전략
 - 각 Deployment에 replicas, resource requests/limits, RollingUpdate 전략을 설정할 수 있는 형태로 YAML을 구성했다.
 - Approval Processing Service처럼 큐 소비량에 따라 부하가 달라지는 서비스는, replicas 값을 조정하여 **수평 확장(Scale-out)** 시나리오를 고려할 수 있다.

11.4.2 장애/확장 시나리오 관점에서의 장점

- Approval Processing Service Pod 일부가 죽더라도, RabbitMQ 큐에는 메시지가 남아 있어 나머지 인스턴스가 처리할 수 있다.
- Employee/Approval/Notification Service는 서로 독립된 Deployment로 존재하므로, 특정 서비스에만 트래픽이 몰리더라도 해당 서비스만 선택적으로 확장할 수 있다.
- NodePort(30080)로 Gateway만 노출하고 나머지 서비스는 클러스터 내부에 숨겨, 외부에서 접근 가능한 공격 표면을 최소화했다.

11.5 통합 효과 및 한계

- 통합 효과
 - JWT로 “사용자·권한”을 표준화하고, RabbitMQ로 “서비스 간 통신”을 안정화하며, Kubernetes로 “배포/운영”을 체계화함으로써 **엔드 투엔드로 일관된 아키텍처**를 구성했다.
 - 과제에서 요구한 기본 기능(결제 생성/조회/승인/반려)을 넘어서, 인증/인가, 큐 기반 비동기 처리, 클라우드 네이티브 배포까지 범위를 포함한다.
- 한계 및 향후 확장 아이디어
 - 현재는 단일 RabbitMQ 인스턴스를 사용하므로, 향후 고가용성을 위해 클러스터링/미러드 큐 구성을 도입할 수 있다.
 - 로그/메트릭/트레이싱(예: ELK, Prometheus, OpenTelemetry)을 연동하면, 장애 분석과 성능 튜닝까지 포함한 “관측 가능성(Observability)” 측면의 완성도가 높아질 것이다.
 - 승인 단계/규칙을 동적으로 구성할 수 있는 Rule 엔진이나, 알림 채널(이메일/Slack 등)을 확장하는 것도 다음 단계 확장 작업으로 고려할 수 있다.